Towards Church theses for finite-state computation: vision and approach

1 Vision

1.1 Broader scientific context

The Church thesis purports all reasonable definitions of what it means to be computable in an intuitive sense. While it is ultimately informal, evidence for it can be found in theorems establishing that different mathematical notions of what it means to be an **algorithm** are equivalent, where equivalent means "they can define the same class of functions $\mathbb{N} \to \{0, 1\}$". Popular models of computations that capture what we call the class of recursive functions popular ones include the **Herbrand-Gödel functionals**, **Turing machines** and the untyped **λ-calculus**.

Very quickly it was determined that studying weaker models of computations is worthwhile, not least because in practice many recursive functions are not really runnable in practice due to their prohibitively high complexity [25]. One way of weakening Turing machines is to require time or space bounds on the computations. Time and space complexity are convenient ways of gauging the efficieny of algorithms, and this naturally led complexity theory to become a topic of great importance in computer science.

On the other hand, the usual way of limiting the power of λ-calculi is by the use of **type systems** which impose a certain discipline on what programs one can write. Although type systems are often motivated by the design of high-level programming languages or logic, in a number of cases they are **normalising**, i.e. that they always terminate. In some cases, the termination argument comes with explicit time bounds. For instance, the simply-typed λ-calculus (STLC) has been shown to characterize the class of elementary-time computable function over finite models [13]. This result is illustrative of a broader class of work in **Implicit Computational Complexity**, the research area that seeks alternative characterizations of complexity classes by programming language or logical means. There is a substantial body of work in this area that achieves those results through the use of typed λ-calculi and linear logic following [10].

In this project, I propose to use these tools to turn our attention to classes of machines that compute their output using only **finite memory**. Such machines are often too weak to be interesting to many complexity theorists, but are rather more studied by **automata theorists**. These finite-state machines, which include automata and transducers, can still model a number of systems of interest while being simple enough so that one may run algorithms to automatically check e.g. safety properties. They also find further applications of their own in other areas such as group theory [7], logic [5] or parsing.

1.2 Past work on "Implicit automata in λ-calculi"

Regular languages are those functions taking as input strings and outputting booleans that are implemented by finite-state automata. They also admit a number of equivalent characterizations, chiefly via MSO formulas, finite monoids and regular expressions. The following remarkable result also links them

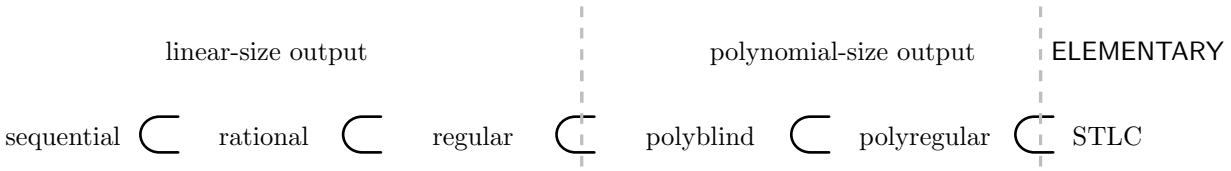


Figure 1: Some transduction classes and their respective growth rates.

to STLC; in this sense, we could propose an analogue of Church thesis saying that regular languages are those boolean-valued maps captured by finite-state computations.

Theorem 1 (Hillebrand-Kanellakis) *$L \subseteq \Sigma^*$ is a regular language if and only if there exists a simple type A and a simply-typed term $t : \mathsf{String}_\Sigma[A] \to \mathsf{Bool}$ recognizing the Church encodings*[1] *$\underline{w}$ of those words $w \in L$.*

The soundness part of the proof (i.e. the right-to-left implication) involves a semantic evaluation of the term t into locally-finite cartesian closed categories. Theorem 1 contrasts heavily with the fact that STLC can encode maps $\mathsf{String}_\Sigma[A] \to \mathsf{String}_\Sigma$ that grows faster than any given tower of exponentials. The key intuition is that Church encodings do not allow fine manipulations like comparisons or subtractions, which is a consequence that those functions, by virtue of Theorem 1 and the compositionality of the λ-calculus, necessarily reflect regular languages. Compositionality and regular language reflection are not enjoyed by a single class of string-to-string functions. In fact, many inequivalent such classes of finite-state functions have been introduced (see e.g. Figure 1 for a non-exhaustive hierarchy of such classes). This means in particular that, contrary to the case of computable functions and Church thesis, sensible notions of string-to-string "finite-state computability" are not determined by "finite-state computability" of their characteristic functions. We therefore might have several analogues of the Church thesis for finite-state machines, for which we ought to collect further technical evidence.

These observations thus motivated a more systematic study of the correspondences between finite-state machines and fragments of (linear) STLC. The corresponding programme, dubbed "Implicit automata in simply-typed λ-calculi", already has already yielded several results [17, 19, 21, 22, 24] which we summarize:

- First, Theorem 1 still holds if we replace the standard Church encodings of strings by the optimised linear variant introduced by Girard [9, §5.3.3] in the context of affine linear logic. That all regular languages are encodable is non-trivial and follows from the Krohn-Rhodes theorem. Furthermore, a correspondence between star-free regular languages is obtained if we make the linear logic non-commutative [21] (and forgo additives).

- Under the same encoding, affine string-to-string functions correspond exactly to the class of regular string functions [19], which admit logical and automata-based characterization. Furthermore, restricting to the non-commutative case (without additive) yields first-order regular functions [24].

- Those later results also extend somewhat to tree-to-tree transducers: the linear functions with additives capture exactly regular tree functions [19]. The fragment without additives however is strictly weaker as it can only capture tree-walking machines [22, 24].

[1] Assuming a base type o, $\mathsf{String} = (o \to o)^{|\Sigma|} \to o \to o$, $\mathsf{Bool} = o^2 \to o$ and $\tau[A]$ means "substitute all instances of o with A in the type τ"; the encoding of words and booleans in String and Bool is then standard, much like in System F.

- Finally allowing duplication of the input in string-to-string function increases power to a well-behaved class of polynomial-growth transducers [17] called polyblind functions, which were introduced in the literature partly for that reason [18].

Those results are not only aesthetically pleasing but also the proofs involved tend to mobilize relatively deep results and techniques from λ-calculus and automata theory such as focusing, determinization of SSTs, the Krohn-Rhodes theorem and denotational semantics inspired by Girard's geometry of interaction. That somewhat involved techniques are involved is somewhat inevitable due to some key differences between the formalisms: for instance, λ-calculus functions are typically trivially closed under composition, which is not the case for transducers. It also means that those investigations also inspired multiple works focused purely on automata theory [4, 6, 18, 20] or λ-calculus [15].

1.3 Aims

A number of challenging open questions regarding implicit automata remain open, including a conjectural characterization of STLC transductions via a variant of collapsible pushdown automata [12] as well as a conjectural characterization of polyregular functions [2] via parsimonious linear λ-calculi [3,14]. Part of the challenge is that the techniques employed in the other cases for soundness could rely on a nice normalisation lemma that made denotational semantics of linear logic without exponentials sufficient to conclude. However, those two conjectures involve arbitrarily-nested exponentials and thus call for figuring out either an approach that interprets full linear logic directly or purely syntactic methods.

Before tackling those more challenging questions, it makes sense to first focus on remaining issues regarding linear λ-terms, expose that material in a cohesive way and test to which extent results and techniques from the disparate worlds of automata and λ-terms can transfer from one setting to another. The plan for this project follows directly this line of thought.

A first phase of the project will consist of an investigation of affine tree-to-tree functions without additives and tree-walking transducer, as well as the analogue of comparison-free functions for tree-to-tree transductions. There are several conjectures that I believe are reasonably easy to tackle and could provide a good way to introduce a researcher familiar with either automata or λ-calculus to results in implicit automata and the relevant techniques.

My more ambitious second objective is to introduce an intuitive variant of System F whose polymorphism is restricted to purely linear types and study its expressivity. Because there exist models of such polymorphism which are locally finite, we know that an analogue of Theorem 1 holds for this system (see [16] for a similar argument). I conjecture that this is also the case for essentially all of the results for linear calculi listed above. This will likely require significant progress when it comes to our collective understanding of the semantics of polymorphism and how to put it to work.

Finally, this project will also allow me to start exploratory work to transfer some automata-theoretic technology to the λ-calculus. Specifically, I propose to seek a well-behaved notion of **origin semantics** [1] for λ-terms operating on Church encodings. Having this would give another tool to refine our analysis of translations between λ-terms and automata, and might be a tool of interest for researchers in λ-calculus. I also believe this will indicate further directions for research in this area that can be pursued after the project's end.

1.4 Broader benefits

This project is about the foundation of theoretical computer science and mathematical in nature. As such, very little non-academic impact is expected.

An important aspect of this project is that it calls for non-trivial expertise in automata theory and semantics, two fields that both belong to the canon of computer science but have rather little overlap aside from higher-order model checking [23] (HOMC), an important area that links advanced

ideas from infinite tree automata and λ-calculi. The tools I shall employ are similar to those of HOMC (chiefly those relying on denotational semantics [11]), although I propose to restrict to a finitary setting with possibly richer outputs and calculi. As such, it is my aim that that audiences from automata theory, semantics of programming language and specialists at the intersection of those communities will benefit from my contributions.

The more basic proof of Theorem 1 is simple enough that could be taught in the introductory automata and computability courses that elect to introduce the λ-calculus. I will be engaging colleagues, in Swansea university and elsewhere, that run courses introducing either the λ-calculus, type theory, computability or the semantics of programming languages (by way of finite state automata, Turing machines, and the λ-calculus) to teach Theorem 1. Further results proven in the course of this project could be introduced in specialist Master or PhD-level courses on transducer theory or linear logic.

2 Approach

2.1 Workpackages

The work of the project is divided into distinct workpackages (WP). Each WP tackles one of the subgoals outline in the Vision section. Most of the work will consist in figuring out proofs of specific conjectures, writing those down and submitting papers to conferences and journals. All workpackages are led by myself as a PI and will be completed by the post-doctoral research assistant (PDRA).

WP1: further results on polynomial tree-to-tree functions (Months 1-6)

As things are now relatively clear when it comes to string-to-string functions in the linear simply-typed λ-calculus in the style of [21], the picture when it comes to tree-to-tree functions is still incomplete. Therefore, I aim to complete it by refining techniques from the string-to-string case.

A first aspect that is under-investigated is the case of polyblind functions. Before even turning to tree functions, the PDRA and I will first write down a clean proof of the following conjecture which we expect to be easy by exploiting [17,24]; this would give an opportunity for the PDRA to familiarize themselves with the main definitions and techniques.

Conjecture 1 *In the non-commutative affine λ-calculus (without additives), string-to-string functions with input duplication are exactly the first-order polyblind functions.*

There are natural generalizations of the "composition by substitution" operator proposed in [18] to trees assuming we enrich the types with "tree with holes". With such a definition, I expect the following generalization of [19, Theorem 1.2] to hold.

Conjecture 2 *In the linear λ-calculus with additives, tree-to-tree functions with input duplication are exactly the tree-to-tree polyblind functions.*

Harder related goals also include finding alternative characterizations of polyblind tree-to-tree functions or results akin to the pebble minimization theorem of [18]. Having these would already justify submitting those results to a top conference (see Dissemination below).

The other direction in which the picture is incomplete is when it comes to the expressiveness of tree-to-tree functions in the absence of additives. So far, I only know of an upper bound; a negative answer to the following would mean it is not tight.

Question 1 *Is there a tree-walking automaton which is not expressible in the linear λ-calculus without additives? What about tree-walking transducers outputting strings?*

Note that so far, those automata models do not speak to tree-to-tree functions. Exploring notions of tree-walking transducers with tree output, planar versions in the sense of [20] and their relation to variation on the Int construction from categorical semantics would be interesting.

WP2: linear impredicative λ-calculi (Months 6-16)

The results in the style of Theorem 1 in the simply-typed setting use a type substitution to crudely emulate a restricted kind of polymorphism. One restriction enforced by the simply-typed setting that is always there is that the polymorphism is predicative, in the sense that universal quantifications can be introduced. We can't just lift that restriction for Theorem 1, lest we allow all definable functions in PA2. However, if we move to linear System F and additionally enforce that types are *purely linear* when using a quantification, then we capture regular languages again. I thus want to study this restricted version of linear System F, which I will call F_ℓ in the rest of this document.

Question 2 *Is* F_ℓ *genuinely more expressive than its predicative counterpart with stratified kinds?*

Question 3 *Does* F_ℓ *admit denotational semantics which are not restrictions of models of linear System F?*

Progress on these questions could then allow me and the PDRA to verify to what extent results from the "implicit automata" programme lift to this new setting. A first objective would be to see if we can somehow generalize [19, Theorem 1.2] to F_ℓ.

WP3: origin semantics for the λ-calculus (Months 12-24)

Origin semantics enriches the interpretation of transducers with information relating positions of the output with their origin in their input, thereby capturing some of the intensional information that is lost when one only considers functions. It can also be defined for other formalisms such as logical interpretations, and be used to refine equivalence between equivalent formalisms. My main goal here is to first develop a reasonable, compositional notion of origin semantics for string-to-string functions in the linear λ-calculus and regular functions. I believe an ad-hoc low-level definition relying on normal form analyses using a monadic translation could work for that purpose, but a goal of this work would be to explore if such a semantics makes sense in a more general higher-order and non-linear context. For instance:

Question 4 *Can we have a sensible origin semantics for string-to-string STLC functions, where an origin is a finitely nested powerset of positions?*

A second step would then to study whether equivalences between, say, regular functions and definable functions string-to-string in the linear λ-calculus ([19, Theorem 1.2]) also holds for the respective origin semantics. Should the PDRA and I manage to answer Question 4 positively, we could also start investigating translations of MSO-set interpretations [8] (with origin semantics) in that setting.

Dissemination The bulk of the work on each workpackage, to be done in the timeframes given above, is to be concluded by a submission to an international conference in the field (such as ICALP, POPL, FSCD or CSL). Me and the PDRA are expected to attend these conferences where our submissions would be accepted. I additionally am planning submissions to relevant specialist workshops such as TLLA and HIGHLIGHTS where we can give talks and invite potential collaborations. After the delineated period of time, the PDRA and I shall also write up journal submissions including full proofs

and details, to be submitted to top theoretical computer science journals such as LMCS or TheoretiCS. More details on the relevant timeline may be found on the Gantt chart at the end of this document.

Swansea's computer science students carry out academic projects in their 3rd and 4th years of study. The project topics can be proposed by students or selected from a list of projects curated by staff. In the second year, I will propose a project topic related to WP3 to excellent Swansea students.

2.2 Planning and risks

As the PI of the NIA award, I will be responsible for the planning and overall project management of the research (see included Gantt chart). In addition, I will hold individual weekly meetings of at least two hours with the PDRA to discuss and make progress on the project. Those meetings will also be used for mentoring sessions. This is a minimum which may be supplemented by other ad-hoc meetings according to my availability and the project needs. I will also invite them to regular weekly research meetings of the computability subgroup during term time.

Three main risks have been identified at this stage, which I will mitigate with appropriate strategies:

- Firstly, not being able to recruit a PDRA of a sufficient level and expertise to meet the project's needs. To mitigate this, I propose to make the offer attractive by offering a post at grade 8 and covering visa fees for international applicants. In addition to relying on HR to disseminate the job advertisement, I will also advertise the positions in relevant specialist mailing lists (such as TYPES-announce, LOGIC or gdr-im) and at in-person external scientific events I attend.

- Second, having the PDRA not being experienced in all areas covered by the project. This is even likely that they won't have deep experience with either denotational semantics or automata theory. To mitigate this, I am planning an extended period for them to review the literature and discussions before starting WP1, which is designed to ease them into the topic at research-level.

- The last main risk is mostly difficulties with coming up with research solutions to some of the problems identified in the workpackages. To mitigate this, we shall review goals in our weekly meetings with the PDRA and switch our focus on areas of the project where progress is more forthcoming if relevant. The questions and conjectures outlined above do not paint an exhaustive picture of what can be attempted on the themes of the different WPs; the alternative conjectures could become focus of our efforts. We shall also keep records of partial progress and put some care in cataloguing examples and counter-examples, to be disseminated at workshops to engage the community on harder problems. We shall also engage Lê Thành Dũng Nguyễn, an expert in the area, via zoom meetings and visit to share progress and planning ideas.

2.3 Resources, deliverable and work environment

This project is mathematical in nature; therefore the resources needed consist of people's time, standard office equipment and travel money to attend conferences. Although some coding will be involved, no specialized IT equipment beyond standard laptops ought to be needed. As a result most of the funds I am requesting would go towards hiring a research associate to work full-time for the duration of the project and support 15% of my time. Additional funds would be allocated to equipment and travel money for conference attendance.

The deliverable will primarly consist of academic papers, to be submitted at top conferences (such as LICS, ICALP, POPL, FoSSaCS or FSCD) and journals (LMCS, Theoretics) in the field.

In terms of facilities, in addition to Swansea University providing us with office space on Bay campus, I will benefit from the rooms reserved for the theoretical computer science group in the Computational Foundry. The PDRA will also be integrated in the theoretical computer science group[2]

[2] `https://swansea-theory.github.io/`

and expected to engage with our various professional development activities, which include a weekly seminar. Our group is also home to other specialists in areas involved in the project: implicit computational complexity in the λ-calculus (Arnold Beckmann), type theory (Anton Setzer, Mukesh Tiwari), and verification (Monika Seisenberger, Anton Setzer and Mukesh Tiwari). The theoretical computer science group also has its own lab within the building that can be used for research sessions and meetings.

References

[1] Mikołaj Bojańczyk. Transducers with origin information. In *International Colloquium on Automata, Languages, and Programming*, pages 26–37. Springer, 2014.

[2] Mikołaj Bojańczyk. Transducers of polynomial growth. In Christel Baier and Dana Fisman, editors, *LICS '22: 37th Annual ACM/IEEE Symposium on Logic in Computer Science, Haifa, Israel, August 2 - 5, 2022*, pages 1:1–1:27. ACM, 2022.

[3] Mikołaj Bojańczyk. Folding interpretations. In *38th Annual ACM/IEEE Symposium on Logic in Computer Science, LICS 2023, Boston, MA, USA, June 26-29, 2023*, pages 1–13. IEEE, 2023.

[4] Mikołaj Bojańczyk, Lê Thành Dũng Nguyễn, and Rafał Stefański. Function spaces for orbit-finite sets. In Karl Bringmann, Martin Grohe, Gabriele Puppis, and Ola Svensson, editors, *51st International Colloquium on Automata, Languages, and Programming, ICALP 2024, July 8-12, 2024, Tallinn, Estonia*, volume 297 of *LIPIcs*, pages 130:1–130:20. Schloss Dagstuhl - Leibniz-Zentrum für Informatik, 2024.

[5] J. Richard Büchi. On a decision method in restricted second order arithmetic. *Proc. Internat. Congr. on Logic, Methodology and Philosophy of Science, 1960*, 1962.

[6] Gaëtan Douéneau-Tabot. Pebble minimization: the last theorems. In Orna Kupferman and Pawel Sobocinski, editors, *Foundations of Software Science and Computation Structures - 26th International Conference, FoSSaCS 2023, Held as Part of the European Joint Conferences on Theory and Practice of Software, ETAPS 2023, Paris, France, April 22-27, 2023, Proceedings*, volume 13992 of *Lecture Notes in Computer Science*, pages 436–455. Springer, 2023.

[7] David BA Epstein. *Word processing in groups*. AK Peters/CRC Press, 1992.

[8] Paul Gallot, Nathan Lhote, and Lê Thành Dũng Nguyên. The structure of polynomial growth for tree automata/transducers and mso set queries, 2025.

[9] Jean-Yves Girard. Linear logic. *Theoretical Computer Science*, 50(1):1–101, January 1987.

[10] Jean-Yves Girard. Light linear logic. *Information and Computation*, 143(2):175–204, 1998.

[11] Charles Grellois and Paul-André Melliès. Finitary semantics of linear logic and higher-order model-checking. In *Mathematical Foundations of Computer Science 2015 - 40th International Symposium, MFCS 2015*, pages 256–268, 2015.

[12] Matthew Hague, Andrzej S. Murawski, C.-H. Luke Ong, and Olivier Serre. Collapsible Pushdown Automata and Recursion Schemes. *ACM Transactions on Computational Logic*, 18(3):25:1–25:42, August 2017.

[13] Gerd G. Hillebrand, Paris C. Kanellakis, and Harry G. Mairson. Database Query Languages Embedded in the Typed Lambda Calculus. *Information and Computation*, 127(2):117–144, June 1996.

[14] Damiano Mazza. Simple Parsimonious Types and Logarithmic Space. In *24th EACSL Annual Conference on Computer Science Logic (CSL 2015)*, pages 24–40, 2015.

[15] Vincent Moreau and Lê Thành Dũng Nguyễn. Syntactically and semantically regular languages of lambda-terms coincide through logical relations, 2023.

[16] Lê Thành Dũng Nguyễn. On the elementary affine lambda-calculus with and without fixed points. In Thomas Seiller and Steffen Jost, editors, *Proceedings Third Joint Workshop on Developments in Implicit Computational complExity and Foundational & Practical Aspects of Resource Analysis, DICE-FOPARA@ETAPS 2019, Prague, Czech Republic, April 6-7, 2019*, volume 298 of *EPTCS*, pages 15–29, 2019.

[17] Lê Thành Dũng Nguyễn. *Implicit automata in linear logic and categorical transducer theory.* PhD thesis, Université Paris XIII (Sorbonne Paris Nord), December 2021.

[18] Lê Thành Dũng Nguyễn, Camille Noûs, and Cécilia Pradic. Comparison-free polyregular functions. In Nikhil Bansal, Emanuela Merelli, and James Worrell, editors, *48th International Colloquium on Automata, Languages, and Programming, ICALP 2021, July 12-16, 2021, Glasgow, Scotland (Virtual Conference)*, volume 198 of *LIPIcs*, pages 139:1–139:20. Schloss Dagstuhl - Leibniz-Zentrum für Informatik, 2021.

[19] Lê Thành Dũng Nguyễn, Camille Noûs, and Cécilia Pradic. Implicit automata in typed λ-calculi II: streaming transducers vs categorical semantics. *CoRR*, abs/2008.01050, 2020.

[20] Lê Thành Dũng Nguyễn, Camille Noûs, and Cécilia Pradic. Two-way automata and transducers with planar behaviours are aperiodic. *CoRR*, abs/2307.11057, 2023.

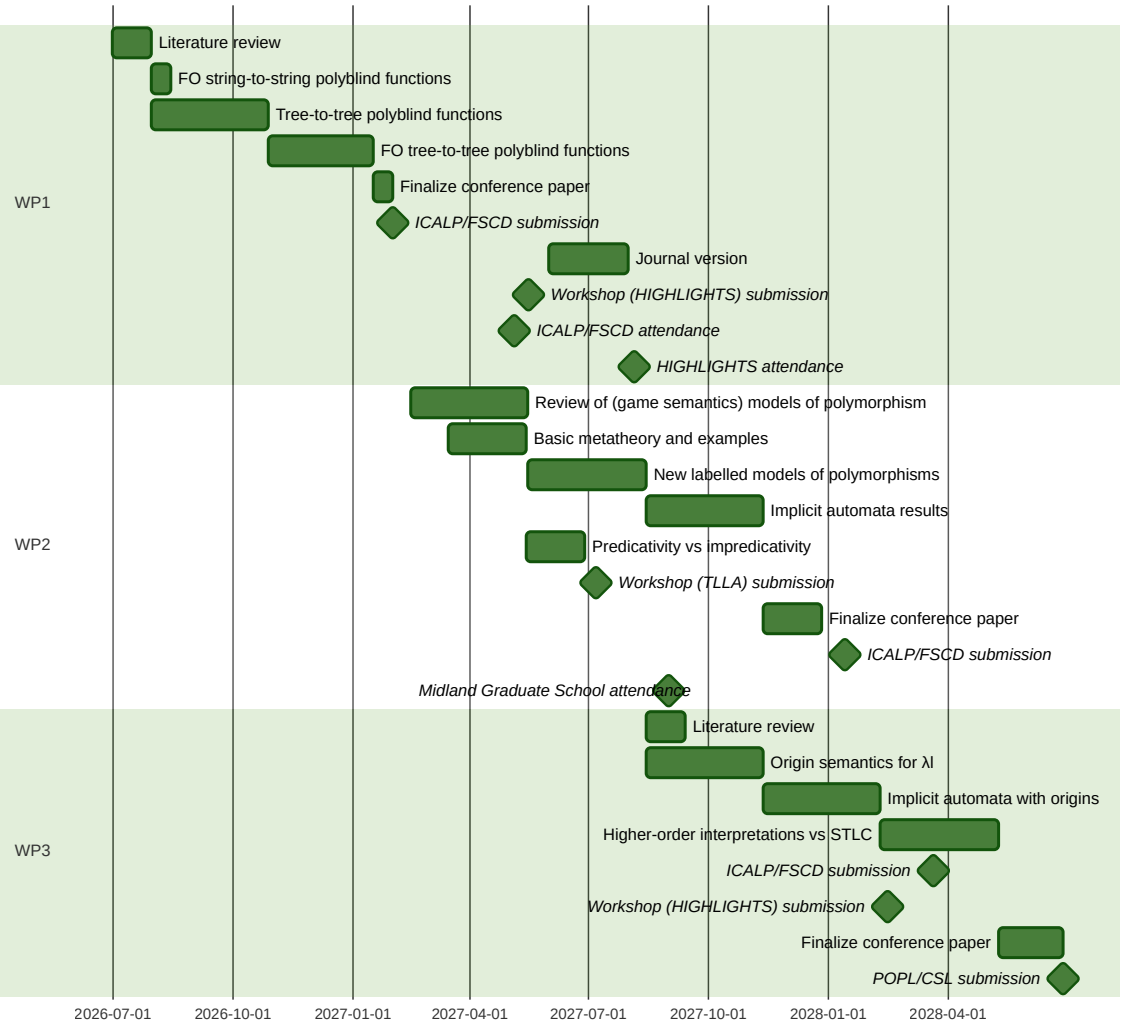
[21] Lê Thành Dũng Nguyễn and Cécilia Pradic. Implicit automata in typed λ-calculi I: aperiodicity in a non-commutative logic. In Artur Czumaj, Anuj Dawar, and Emanuela Merelli, editors, *47th International Colloquium on Automata, Languages, and Programming, ICALP 2020, July 8-11, 2020, Saarbrücken, Germany (Virtual Conference)*, volume 168 of *LIPIcs*, pages 135:1–135:20. Schloss Dagstuhl - Leibniz-Zentrum für Informatik, 2020.

[22] Lê Thành Dũng Nguyễn and Gabriele Vanoni. (Almost) affine higher-order tree transducers. *CoRR*, abs/2402.05854, 2024.

[23] C.-H. Luke Ong. On Model-Checking Trees Generated by Higher-Order Recursion Schemes. In *21st Annual IEEE Symposium on Logic in Computer Science (LICS'06)*, pages 81–90, Seattle, WA, USA, 2006. IEEE.

[24] Cécilia Pradic and Ian Price. Implicit automata in λ-calculi III: affine planar string-to-string functions. *Electronic Notes in Theoretical Informatics and Computer Science*, 4, 2024.

[25] Alasdair Urquhart. Von neumann, gödel and complexity theory. *Bulletin of Symbolic Logic*, 16(4):516–530, 2010.


WP1
Literature review
FO string-to-string polyblind functions
Tree-to-tree polyblind functions
FO tree-to-tree polyblind functions
Finalize conference paper
ICALP/FSCD submission
Journal version
Workshop (HIGHLIGHTS) submission
ICALP/FSCD attendance
HIGHLIGHTS attendance
WP2
Review of (game semantics) models of polymorphism
Basic metatheory and examples
New labelled models of polymorphisms
Implicit automata results
Predicativity vs impredicativity
Workshop (TLLA) submission
Finalize conference paper
ICALP/FSCD submission
Midland Graduate School attendance
WP3
Literature review
Origin semantics for λI
Implicit automata with origins
Higher-order interpretations vs STLC
ICALP/FSCD submission
Workshop (HIGHLIGHTS) submission
Finalize conference paper
POPL/CSL submission
2026-07-01
2026-10-01
2027-01-01
2027-04-01
2027-07-01
2027-10-01
2028-01-01
2028-04-01